Identifying Key Entities in Recipe Data Report:

1. Business Objective

The goal of this project is to train a Named Entity Recognition (NER) model using Conditional Random Fields (CRF) to extract key entities from recipe data. The model classifies words into predefined categories such as ingredients, quantities, and units, enabling the creation of a structured database of recipes and ingredients. This can be used in recipe management systems, dietary tracking apps, or e-commerce platforms.

2. Data Description

The data is in JSON format, representing a structured recipe ingredient list with NER labels. It contains two fields:

- input: A raw ingredient list from a recipe.
- pos: Corresponding part-of-speech (POS) tags or NER labels, identifying quantities, ingredients, and units.

2.1.3 Describe the dataframe [3 marks]

Print first five rows of dataframe along with dimensions. Display the information of dataframe

```
[ ] # display first five rows of the dataframe - df
    df.head()
```

	input	pos
0	6 Karela Bitter Gourd Pavakkai Salt 1 Onion 3 tablespoon Gram flour besan 2 teaspoons Turmeric powder Haldi Red Chilli Cumin seeds Jeera Coriander Powder Dhania Amchur Dry Mango Sunflower Oil	quantity ingredient ingredient ingredient ingredient ingredient quantity ingredient quantity unit ingredient ingredient ingredient quantity unit ingredient
1	2-1/2 cups rice cooked 3 tomatoes teaspoons BC Belle Bhat powder 1 teaspoon chickpea lentils 1/2 cumin seeds white urad dal mustard green chilli dry red 2 cashew or peanuts 1-1/2 tablespoon oil asafoetida	quantity unit ingredient ingredient quantity ingredient unit ingredient ingredient ingredient ingredient quantity unit ingredient ingredient quantity ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient

Print the first five rows of train_df and val_df

3.1.2 Print the first five rows of train_df and val_df [1 marks]

```
[ ] # print the first five rows of train_df
    train_df.head()
```

	input	pos	input_tokens	pos_tokens	input_length	pos_length
179	250 grams Okra Oil 1 Onion finely chopped Tomato Grated teaspoon Ginger 2 Garlic Finely 1/2 Cumin seeds 1/4 Teaspoon asafoetida cup cottage cheese pinched coriander powder mango red chilli turmeric	quantity unit ingredient ingredient quantity ingredient ingredient ingredient ingredient ingredient unit ingredient quantity ingredient ingredient quantity ingredient ingredient quantity unit ingredient unit ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient	[250, grams, Okra, Oil, 1, Onion, finely, chopped, Tomato, Grated, teaspoon, Ginger, 2, Garlic, Finely, 1/2, Cumin, seeds, 1/4, Teaspoon, asafoetida, cup, cottage, cheese, pinched, coriander, powder, mango, red, chilli, turmeric]	[quantity, unit, ingredient, ingredient, quantity, ingredient, ingredient, ingredient, ingredient, ingredient, unit, ingredient, quantity, ingredient, ingredient, quantity, ingredient, ingredient, quantity, unit, ingredient, unit, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient]	31	31
57	200 grams Paneer Homemade Cottage Cheese 2 Potato Aloo Bay leaf tej patta Dry Red Chilli 1 tablespoon Panch Phoran Masala roasted and powdered Tomato big sized teaspoon Turmeric powder Haldi Cumin seeds Jeera Ginger grated Salt 1/2 Sugar Sunflower Oil	quantity unit ingredient ingredient ingredient ingredient quantity ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient quantity unit ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient	[200, grams, Paneer, Homemade, Cottage, Cheese, 2, Potato, Aloo, Bay, leaf, tej, patta, Dry, Red, Chilli, 1, tablespoon, Panch, Phoran, Masala, roasted, and, powdered, Tomato, big, sized, teaspoon, Turmeric, powder, Haldi, Cumin, seeds, Jeera, Ginger, grated, Salt, 1/2, Sugar, Sunflower, Oil]	[quantity, unit, ingredient, ingredient, ingredient, ingredient, quantity, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, quantity, unit, ingredient]	41	41
112	500 grams Cabbage Patta Gobi Muttaikose 1 teaspoon Mustard seeds 1-1/2 White Urad Dal Split sprig Curry leaves Green Chilli 1/4 cup Fresh coconut Salt	quantity unit ingredient ingredient ingredient ingredient quantity unit ingredient ingredient quantity ingredient ingredient ingredient ingredient unit ingredient ingredient ingredient ingredient quantity unit ingredient ingredient ingredient	[500, grams, Cabbage, Patta, Gobi, Muttaikose, 1, teaspoon, Mustard, seeds, 1-1/2, White, Urad, Dal, Split, sprig, Curry, leaves, Green, Chilli, 1/4, cup, Fresh, coconut, Salt]	[quantity, unit, ingredient, ingredient, ingredient, ingredient, quantity, unit, ingredient, ingredient, quantity, ingredient, ingredient, ingredient, ingredient, unit, ingredient, ingredient, ingredient, ingredient, quantity, unit, ingredient, ingredient, ingredient]	25	25
218	500 grams Fresh Figs 1/4 cup Lemon juice 1 teaspoon zest 2 Red Chilli flakes 1/2 Honey Brown Sugar (Demerara Sugar)	quantity unit ingredient ingredient quantity unit ingredient ingredient quantity unit ingredient quantity ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient	[500, grams, Fresh, Figs, 1/4, cup, Lemon, juice, 1, teaspoon, zest, 2, Red, Chilli, flakes, 1/2, Honey, Brown, Sugar, (Demerara, Sugar)]	[quantity, unit, ingredient, ingredient, quantity, unit, ingredient, ingredient, quantity, unit, ingredient, quantity, ingredient, ingredient, ingredient, ingredient, ingredient, quantity, ingredient, ingredient, ingredient, ingredient, ingredient]	21	21
40	2 cups Water 1 teaspoon Tea leaves 1/4 Milk 10 Saffron strands	quantity unit ingredient quantity unit ingredient ingredient quantity ingredient quantity ingredient ingredient	[2, cups, Water, 1, teaspoon, Tea, leaves, 1/4, Milk, 10, Saffron, strands]	[quantity, unit, ingredient, quantity, unit, ingredient, ingredient, quantity, ingredient, quantity, ingredient, ingredient]	12	12

4. Exploratory Data Analysis (EDA)

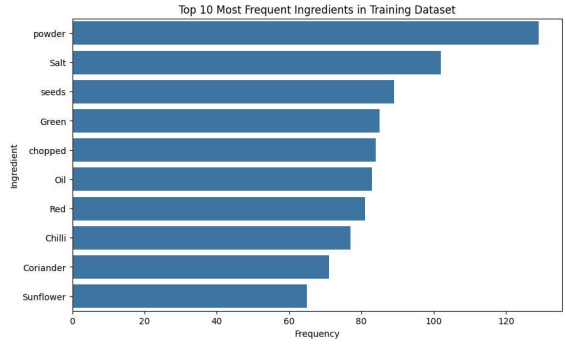
- **Token Flattening**: Functions were defined to flatten the lists of input tokens and POS tags.
- **Token Extraction and Validation**: Tokens were extracted and validated to ensure the number of input tokens matched the number of POS tags.
- **Token Categorization**: Tokens were categorized into ingredients, units, and quantities.
- **Frequency Analysis**: The top 10 most frequent ingredients and units were identified and visualized using bar plots.
  - **Top 10 Ingredients**: "powder", "Salt", "seeds", "Green", "chopped", "Oil", "Red", "Chilli", "Coriander", and "Sunflower".
  - **Top 10 Units**: "teaspoon", "cup", "tablespoon", "grams", "tablespoons", "inch", "cups", "sprig", "cloves", and "teaspoons".

Perform EDA analysis

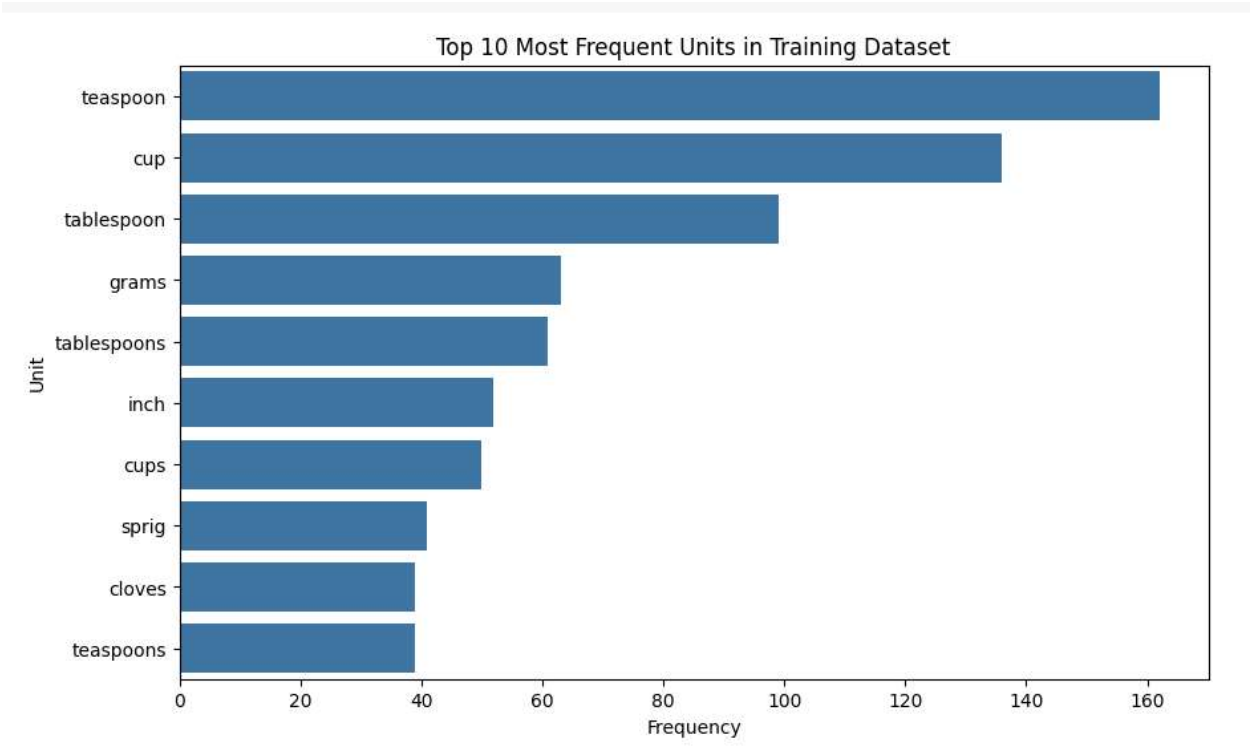
4.6 *Perform EDA analysis* [5 marks]

Plot the bar plots for ingredients and units and provide the insights for training dataset

```
[ ] # plot the top frequent ingredients in training data
    plot_top_items(ingredients, "ingredient", dataset_name)
```




```
<Figure size 640x480 with 0 Axes>
```



5. Data Preprocessing

- **Feature Engineering**: Features were engineered to aid in entity recognition, including context, numeric values, and keyword matching.
- **Data Splitting**: The dataset was split into training and validation sets (70:30 ratio).

6. Model Training

- **CRF Model Initialization**: A Conditional Random Fields (CRF) model was initialised with specific hyperparameters.
- **CRF Model Training**: The CRF model was trained on the training data.

7. Model Evaluation

- **Training Data Evaluation**: The model's performance on the training data was evaluated using a classification report and confusion matrix.
- **Validation Data Evaluation**: The model's performance on the validation data was assessed, and error analysis was conducted to identify common misclassifications.

7.2 *Evaluation of Training Dataset using CRF model* [4 marks]

Evaluate on training dataset using CRF by using flat classification report and confusion matrix

```
[ ] # evaluate on the training dataset
    crf.fit(X_train, y_train)

    y_pred_train = crf.predict(X_train)

    from sklearn_crfsuite import metrics
    print(metrics.flat_classification_report(y_train, y_pred_train, digits=3))
```

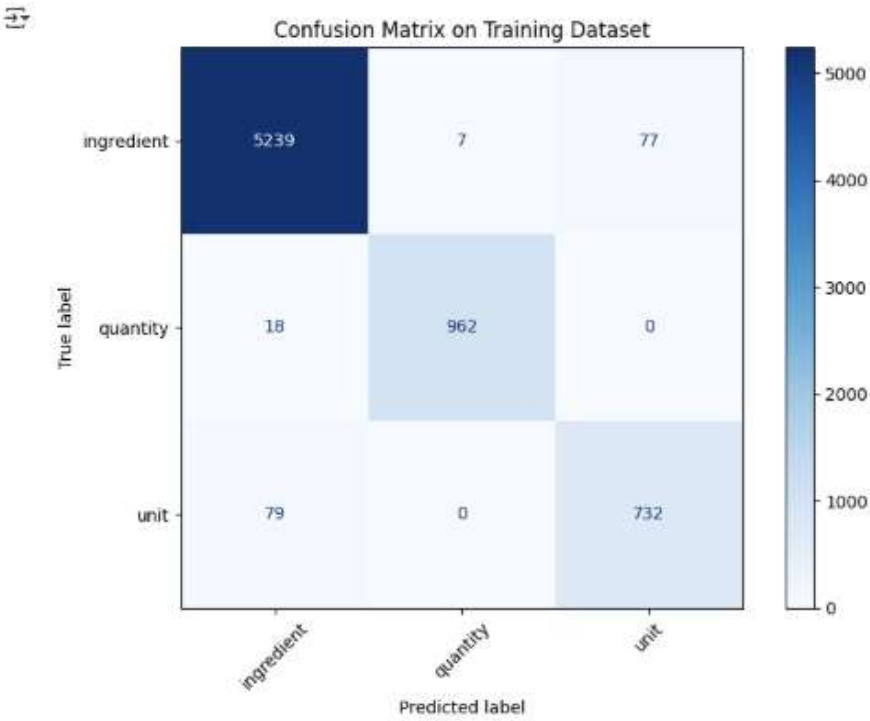
```
              precision    recall  f1-score   support

  ingredient      0.982     0.984     0.983      5323
    quantity      0.993     0.982     0.987       980
        unit      0.905     0.903     0.904       811

    accuracy                          0.975      7114
   macro avg      0.960     0.956     0.958      7114
weighted avg      0.975     0.975     0.975      7114
```

```
[ ] # specify the flat classification report by using training data for evaluation
    report = metrics.flat_classification_report(y_train, y_pred_train, digits=3)
    print(report)
```

```
              precision    recall  f1-score   support

  ingredient      0.982     0.984     0.983      5323
    quantity      0.993     0.982     0.987       980
        unit      0.905     0.903     0.904       811

    accuracy                          0.975      7114
   macro avg      0.960     0.956     0.958      7114
weighted avg      0.975     0.975     0.975      7114
```

Confusion Matrix:

```
[ ] # create a confusion matrix on training datset
    from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
    import matplotlib.pyplot as plt
    import numpy as np

    # Flatten the lists of true and predicted labels
    y_true_flat = [label for seq in y_train for label in seq]
    y_pred_flat = [label for seq in y_pred_train for label in seq]

    # Get unique labels from both true and predicted sets
    labels = sorted(list(set(y_true_flat + y_pred_flat)))

    # Compute confusion matrix
    cm = confusion_matrix(y_true_flat, y_pred_flat, labels=labels)

    # Display the confusion matrix
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
    fig, ax = plt.subplots(figsize=(8, 6))
    disp.plot(ax=ax, cmap='Blues', xticks_rotation=45)
    plt.title("Confusion Matrix on Training Dataset")
    plt.tight_layout()
    plt.show()
```



Prediction and Model Evaluation:

8 Prediction and Model Evaluation [3 marks]

8.1 *Predict and Evaluate the CRF model on validation set* [3 marks]

Evaluate the metrics for CRF model by using flat classification report and confusion matrix

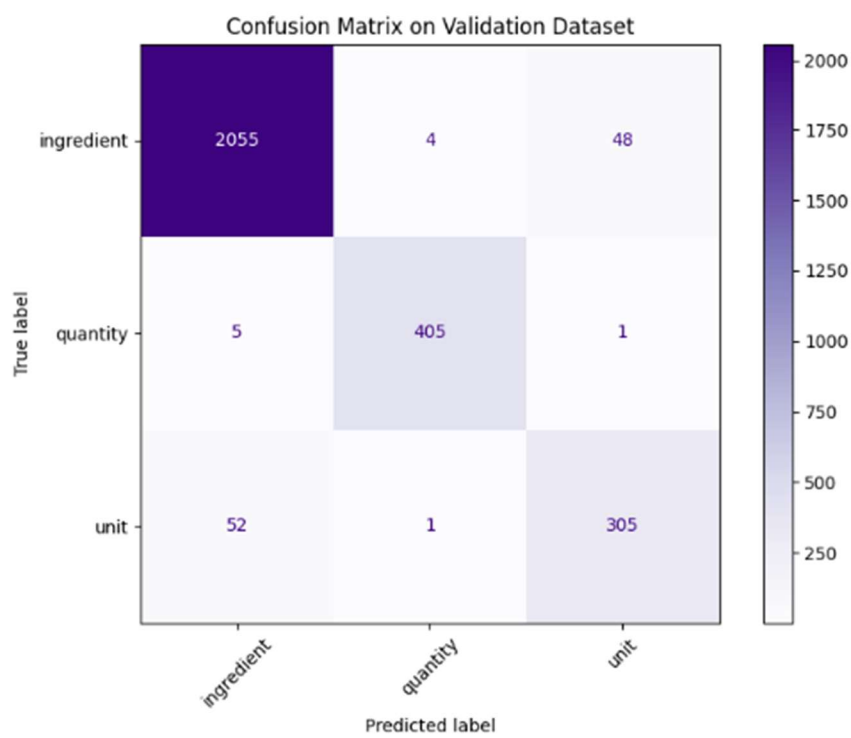
```
[ ] # predict the crf model on validation dataset
    y_pred_val = crf.predict(X_val)
```

```
[ ] # specify flat classification report

    from sklearn_crfsuite import metrics

    # Generate and print classification report
    report_val = metrics.flat_classification_report(y_val, y_pred_val, digits=3)
    print(report_val)
```

```
              precision    recall  f1-score   support

  ingredient      0.973     0.975     0.974      2107
    quantity      0.988     0.985     0.987       411
        unit      0.862     0.852     0.857       358

    accuracy                          0.961      2876
   macro avg      0.941     0.938     0.939      2876
weighted avg      0.961     0.961     0.961      2876
```

<u>Confusion Matrix On Validation dataset:</u>

```python
# create a confusion matrix on validation dataset

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Flatten the lists of true and predicted labels
y_val_flat = [label for seq in y_val for label in seq]
y_pred_val_flat = [label for seq in y_pred_val for label in seq]

# Get unique labels (ensure consistent ordering)
labels = sorted(list(set(y_val_flat + y_pred_val_flat)))

# Create the confusion matrix
cm_val = confusion_matrix(y_val_flat, y_pred_val_flat, labels=labels)

# Display the confusion matrix
disp = ConfusionMatrixDisplay(confusion_matrix=cm_val, display_labels=labels)
fig, ax = plt.subplots(figsize=(8, 6))
disp.plot(ax=ax, cmap='Purples', xticks_rotation=45)
plt.title("Confusion Matrix on Validation Dataset")
plt.tight_layout()
plt.show()
```



8. Error Analysis

- **Detailed Error Analysis**: Errors were analysed to understand the model's failure points, considering factors like class weights and token context.
- **Error Categorization**: Errors were categorised and analysed to identify patterns.

9.1.3 Create dataframe from error_data and print overall accuracy [1 marks]

Change error_data into dataframe and then use it to illustrate the overall accuracy of validation data

```
# Create DataFrame and Print Overall Accuracy


# Create a DataFrame from error_data
error_df = pd.DataFrame(error_data)

# Calculate overall accuracy
total_tokens = sum(len(labels) for labels in y_val)  # Total number of tokens in validation set
incorrect_tokens = len(error_df)  # Number of misclassified tokens
accuracy = (total_tokens - incorrect_tokens) / total_tokens

print(f"Overall Accuracy: {accuracy:.2%}")  # Print accuracy as a percentage

# Display the DataFrame
display(error_df)
```

```
Overall Accuracy: 96.14%
```

	token	true_label	predicted_label	previous_token	next_token
0	liter	unit	ingredient	2	Milk
1	coconut	ingredient	unit	Fresh	grated
2	inch	unit	ingredient	nuts	Cinnamon
3	pasta	ingredient	unit	tomato	sauce
4	cloves	ingredient	unit	3	garlic
...	...	...	...	...	...
106	chips	ingredient	unit	Chocolate	dark
107	soft	ingredient	unit	cream	paneer
108	on	ingredient	quantity	sprinkle	top
109	top	ingredient	unit	on	<END>
110	coconut	ingredient	unit	1	grated

111 rows × 5 columns

9. Results and Insights

- The CRF model achieved high overall accuracy in extracting key entities from recipe data.
- The model performed well on ingredient and quantity extraction but showed some challenges with unit recognition.
- Error analysis revealed common misclassification patterns, such as confusion between ingredients and units, and the impact of context on entity recognition.

10. Conclusion

A CRF model was successfully developed to extract key entities from recipe data. The model achieved high overall accuracy, with room for improvement in specific cases. Future work could focus on refining features and handling challenging scenarios. This project demonstrates the effectiveness of CRF for recipe ingredient extraction, with potential applications in various culinary domains.